

제6회 K-인공지능 제조데이터 분석 경진대회

X-ray 이물 검출 모델 6종을 비교하고 일반화·강건성까지 검증해 베스트 모델을 선정하다

영상 기반 완제품 이물질 탐지 및 AI 미탐지 조건 분석

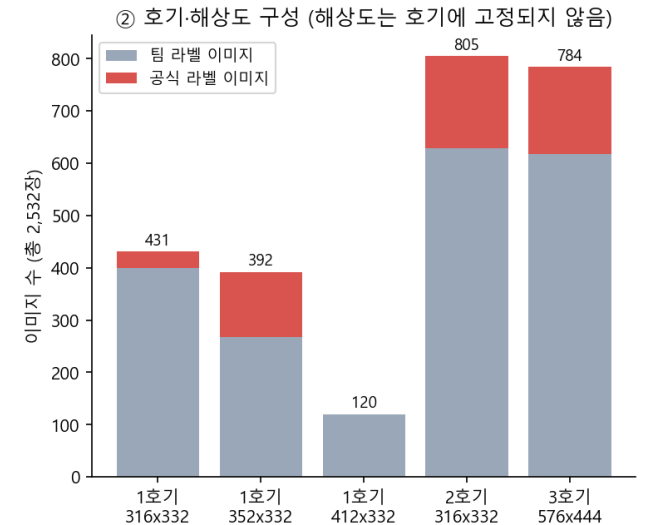
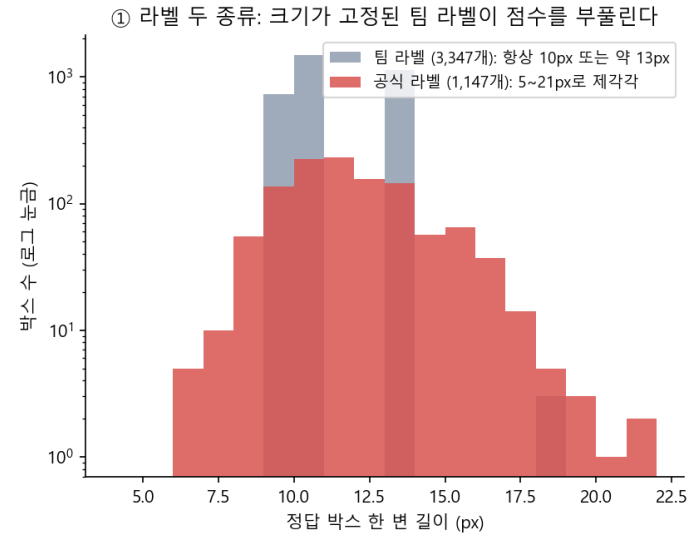
팀명: [확정 필요]

허민엽 · 정연창 · 이총

문제와 데이터

약 10px의 작은 점(금속구)을 찾는 검출 문제, 이미지는 모두 NG 판정

- X-ray 이물 검출기 영상 **2,532장**, 박스 4,494개
- 호기 3종, 해상도 4종 (316x332 ~ 576x444)
- train 1,767 / val 369 / test 396장 (연속 촬영 묶음 단위, 시드 42)
- **라벨이 두 종류**: 공식 500장은 사람이 그려 크기가 제각각(5~21px), 팀 라벨 2,032장은 클릭 중심에 고정 크기 네모(**10·13px**)를 자동 생성
- 정상 이미지가 없다(빈 라벨 105장은 제품이 반만 찍힌 사진)
- 장비가 그린 색 표시는 전처리로 제거(다음 슬라이드)

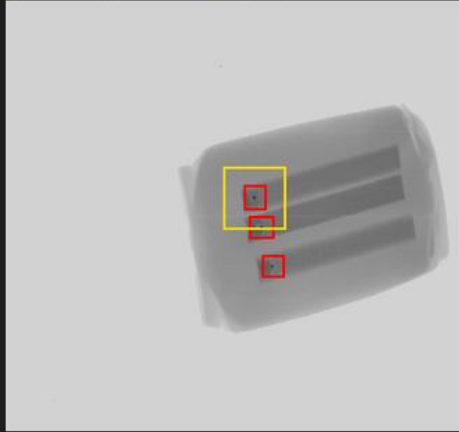


전처리: 색 윤곽만 지우고 점은 남긴다

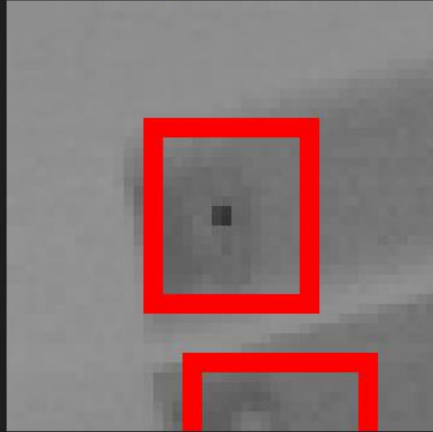
장비가 그린 색 사각형을 주변 회색 5x5 평균으로 메운 뒤 회색조 PNG로 저장

전처리: 장비가 그린 색 윤곽(원본 BMP)을 주변 회색 5x5 평균으로 메운다. 점(가운데)은 그대로 남는다

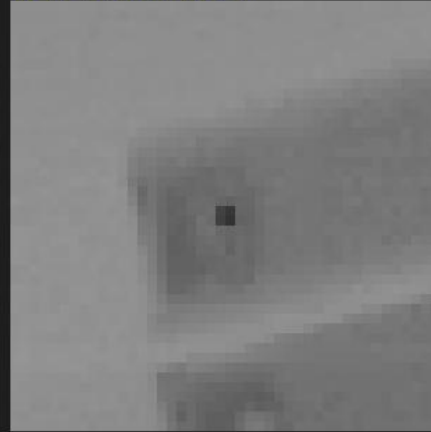
1호기 원본 (노랑 = 확대 영역)



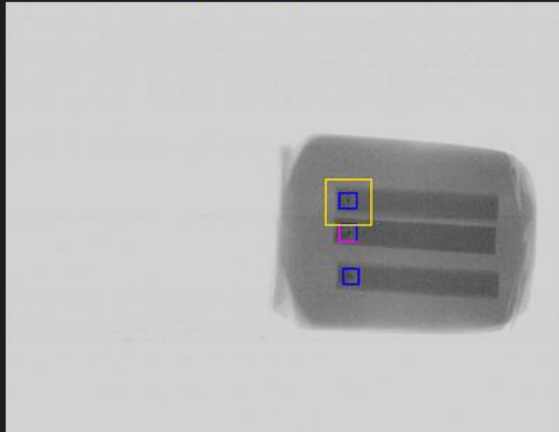
원본 확대: 색 윤곽



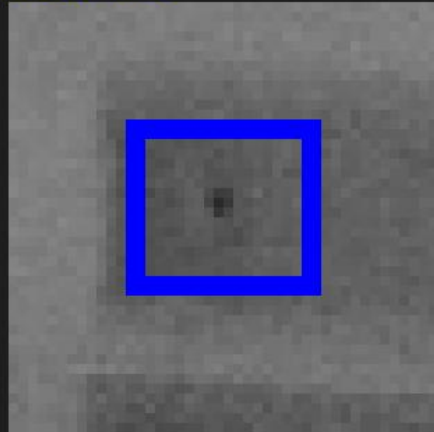
전처리 후 (data/의 PNG)



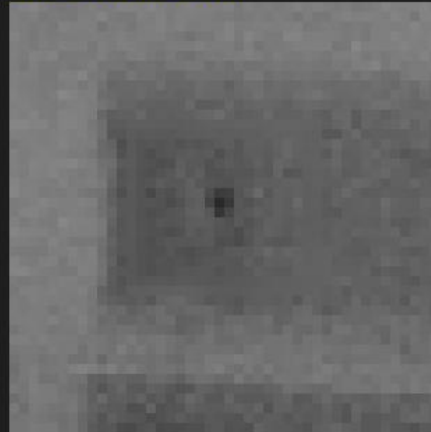
3호기 원본 (노랑 = 확대 영역)



원본 확대: 색 윤곽



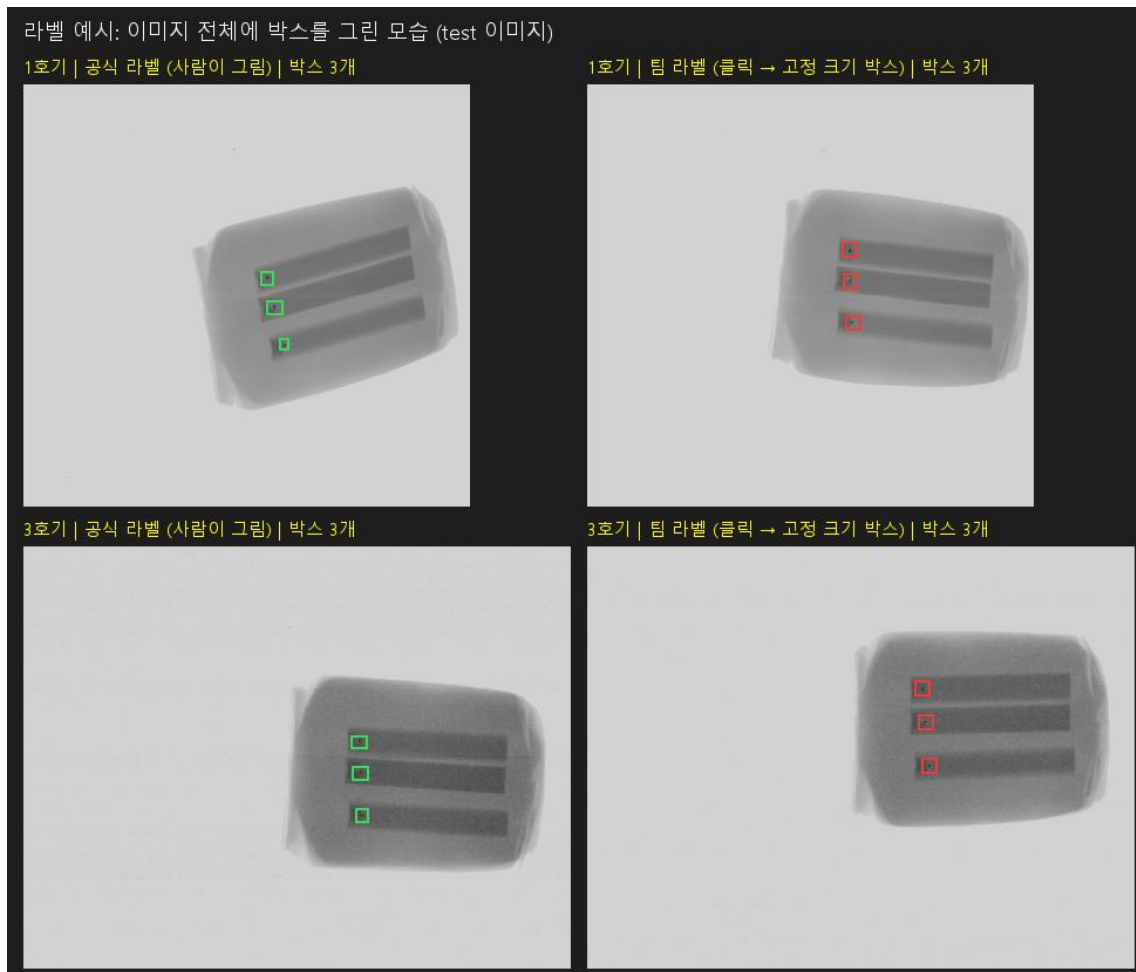
전처리 후 (data/의 PNG)



- 색 표시는 결함 둘레의 가는 사각형 윤곽(장당 128~660픽셀), 점은 그대로 남는다
- 지우는 이유: 공식 라벨 중심의 99.8%가 색 박스 안 → 지우지 않으면 모델이 "색 네모 찾기"를 배운다
- 원본 BMP에 같은 방법을 다시 적용하면 **2,532장 모두 픽셀 단위로 일치**(검증 코드 포함)
- 코드는 이층의 브랜치(scripts/preprocess/)에 있고 main 병합이 필요하다

라벨링: 사람이 직접 만든 박스

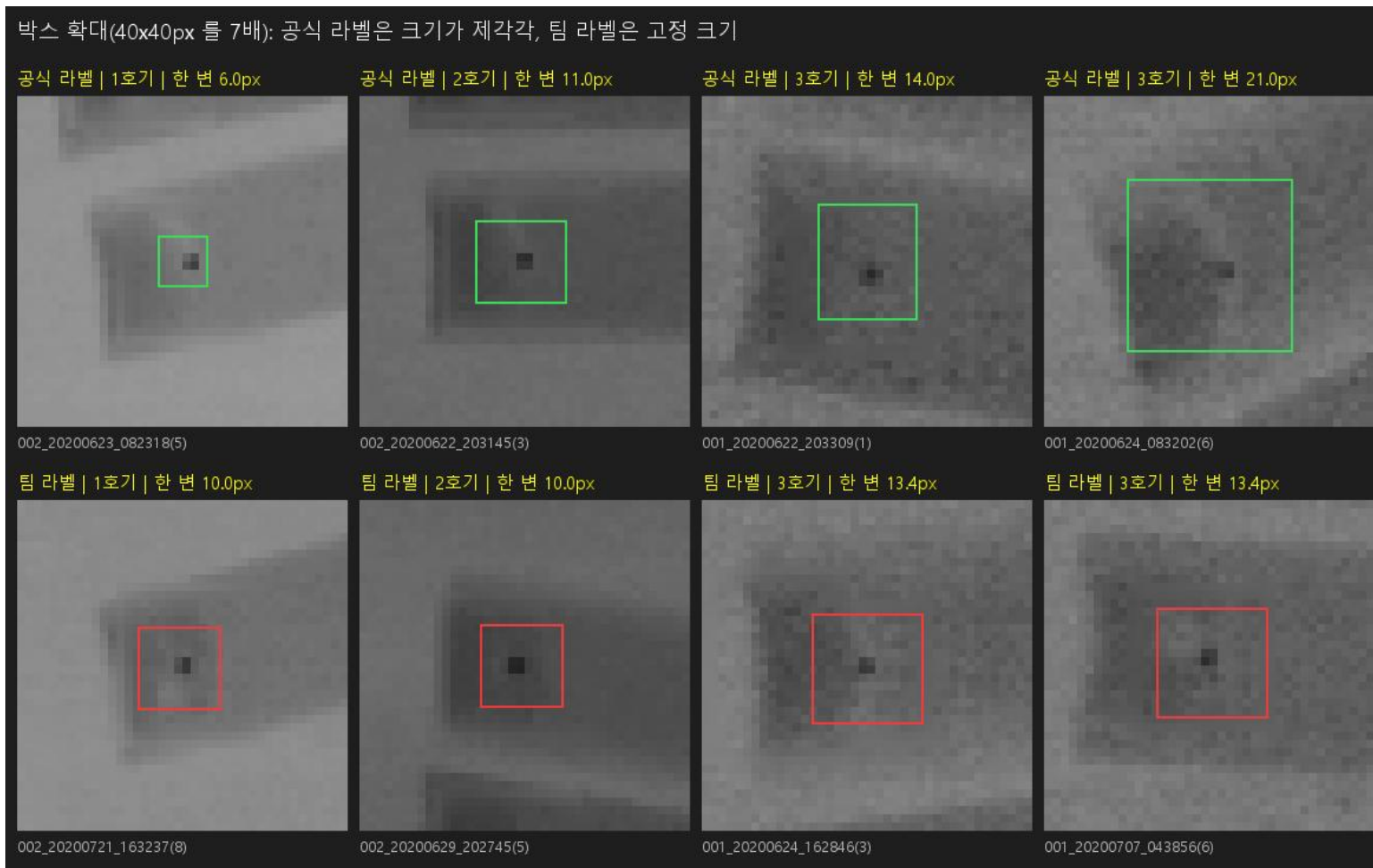
공식 라벨 500장(사람이 그림)과 팀 라벨 2,032장(점을 클릭하면 박스가 자동 생성)



- 공식 라벨 500장(박스 1,147개): KAMP 제공, 사람이 박스를 그려 크기가 제각각(5~21px)
- 팀 라벨 2,032장(박스 3,347개): 팀원 3명(674·678·680장)이 Label Studio에서 금속구 **한가운데를** 클릭하면 1초 뒤 박스가 자동 생성
- 연속 사진은 점이 몇 px씩 움직여 매 장 새로 찍었고, 못 찾으면 건너뛰어 빈 라벨로 처리
- 점 자체(지름 몇 px의 검은 점)를 찍고, 흐릿한 얼룩은 찍지 않는다

박스 확대: 공식은 제각각, 팀은 고정 크기

팀 라벨은 클릭 좌표를 중심으로 한 번 10px × (이미지 높이 / 332)의 정사각형



- 공식 라벨 한 번 6~21px(호기별 중앙값 10·11·14px)
- 팀 라벨: 1·2호기 10px, 3호기 13.4px. 공식 라벨 한 번 중앙값에 맞춘 값
- 고정 크기라 사람마다 크기가 달라지지 않지만 결함 크기 정보가 없다
- → 모델이 같은 크기만 내도 IoU가 잘 맞아 점수가 부풀려진다

3호기 팀 박스 1,129개 중 3개(한 이미지)만 17.7x18.6px로 예외이고 나머지는 13.4px이다.

데이터셋 구성 ①: 분할과 라벨

이미지 2,532장, 박스 4,494개, 모든 이미지에 라벨 파일이 있다(빈 라벨 105장)

구분	이미지	박스	빈 라벨 이미지	공식 라벨 이미지 (박스)	팀 라벨 이미지 (박스)	촬영 묶음(burst)
train	1,767	3,225	59	356 (815)	1,411 (2,410)	382
val	369	606	19	59 (139)	310 (467)	82
test	396	663	27	85 (193)	311 (470)	83
합계	2,532	4,494	105	500 (1,147)	2,032 (3,347)	547

박스 한 변의 길이(px): 최소 / 중앙값 / 최대 (박스 수)

라벨 출처	1호기	2호기	3호기
공식 라벨 (사람이 그림, 500장)	6 / 10 / 14 (404)	7 / 11 / 18 (376)	8 / 14 / 21 (367)
팀 라벨 (고정 크기 자동 생성, 2,032장)	10 / 10 / 10 (1,143)	10 / 10 / 10 (1,075)	13.4 / 13.4 / 18.6 (1,129)

- 팀 라벨은 3명이 674~678~680장씩 작성. 촬영 시기 6~9월(6월 447장, 7월 841장, 8월 184장, 9월 1,060장). 이미지는 회색조 PNG, 라벨은 YOLO txt(클래스 1개)

데이터셋 구성 ②: 호기·해상도별

해상도는 호기별로 고정이고, 빈 라벨 105장 중 104장이 1호기 412x332다

호기	이미지	train	val	test	박스	빈 라벨	공식 라벨 이미지	팀 라벨 이미지
1호기	943	672	134	137	1,547	104	156	787
2호기	805	573	102	130	1,451	1	177	628
3호기	784	522	133	129	1,496	0	167	617
합계	2,532	1,767	369	396	4,494	105	500	2,032

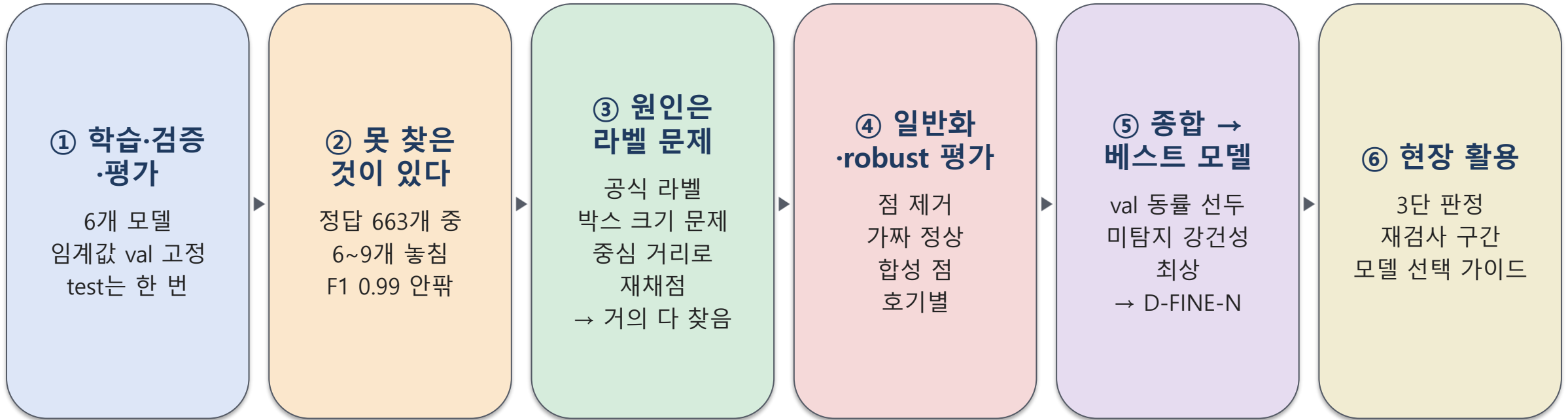
호기 x 해상도

호기	해상도	이미지	train / val / test	박스	빈 라벨	공식 라벨 이미지 (박스)	팀 라벨 이미지 (박스)
1호기	316x332	431	329 / 40 / 62	431	0	32 (32)	399 (399)
1호기	352x332	392	273 / 73 / 46	1,100	0	124 (372)	268 (728)
1호기	412x332	120	70 / 21 / 29	16	104	0 (0)	120 (16)
2호기	316x332	805	573 / 102 / 130	1,451	1	177 (376)	628 (1,075)
3호기	576x444	784	522 / 133 / 129	1,496	0	167 (367)	617 (1,129)
합계	4종	2,532	1,767 / 369 / 396	4,494	105	500 (1,147)	2,032 (3,347)

출처: data/manifest.csv. 파일명 접두 001/002는 호기가 아니다. 호기는 manifest의 machine 열이다.

실험 흐름

학습·평가 → 못 찾은 것의 원인 → 일반화·robust 평가 → 베스트 모델



- 모델 6개: 베이스라인 YOLOv3-tiny, Faster R-CNN(ResNet-50, MobileNetV3), 팀원 모델 YOLO26n · RT-DETR-L · D-FINE-N
- 모든 모델을 **같은 평가 코드**로 채점하고, 임계값은 val에서 정해 고정, test는 한 번만 본다

① 학습 · 검증 · 평가

지금 있는 데이터로 6개 모델을 같은 조건에서 학습하고 평가했다

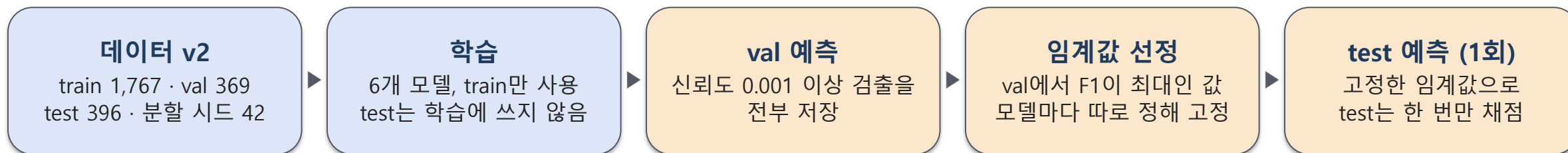
모델	입력	학습 설정	val 임계값
YOLOv3-tiny (베이스라인)	640	COCO 사전학습, 100 epoch	0.06
Faster R-CNN ResNet-50	640	COCO 사전학습, 20 epoch(선택 7)	0.95
Faster R-CNN MobileNetV3	640	COCO 사전학습, 20 epoch(선택 18)	0.93
YOLO26n (팀원)	1024	COCO 사전학습, 40 epoch	0.42
RT-DETR-I (팀원)	640	COCO 사전학습, 최대 30 epoch(계획)	0.74
D-FINE-N (팀원)	640	학습 설정 기록 [확인 필요]	0.66

- 같은 데이터·분할, 같은 평가 코드
- 임계값은 모델마다 **val**에서 **F1 최대**로 정해 고정
- **test**는 한 번만 평가
- 팀원 모델은 가중치를 받아 같은 코드로 다시 평가(임계값도 다시 선정)

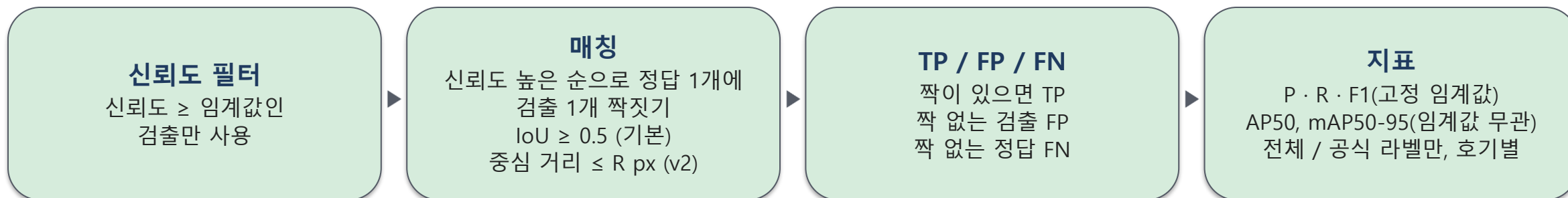
평가 아키텍처: 같은 코드로 모든 모델을 채점

임계값은 val에서 정해 고정하고, test는 한 번만 본다

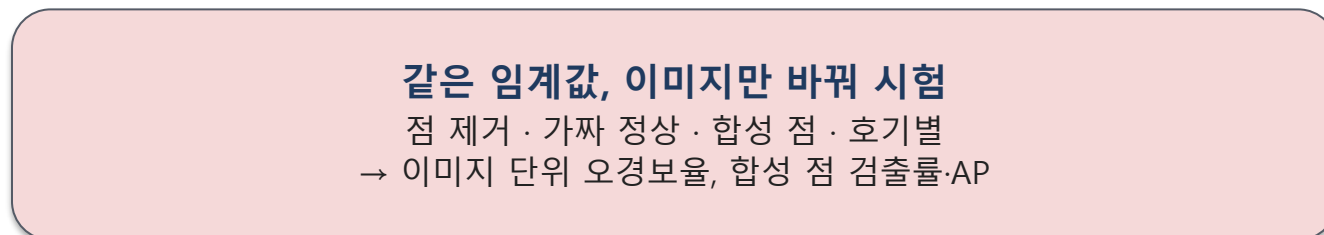
① 예측과 임계값 선정



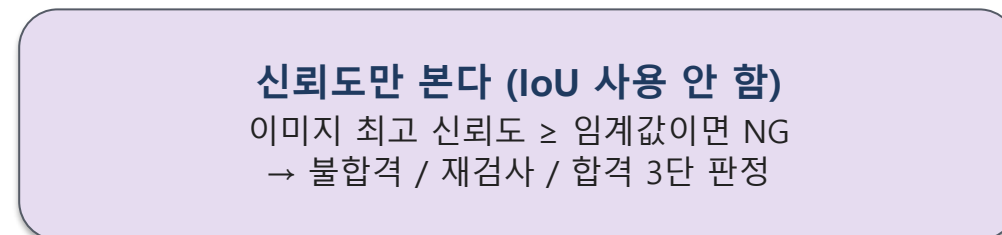
② 채점 (정답 있음): 신뢰도와 IoU를 둘 다 본다



③ 일반화·robust 평가



현장 판정 (정답 없음)



결과: 몇 개를 못 찾았다

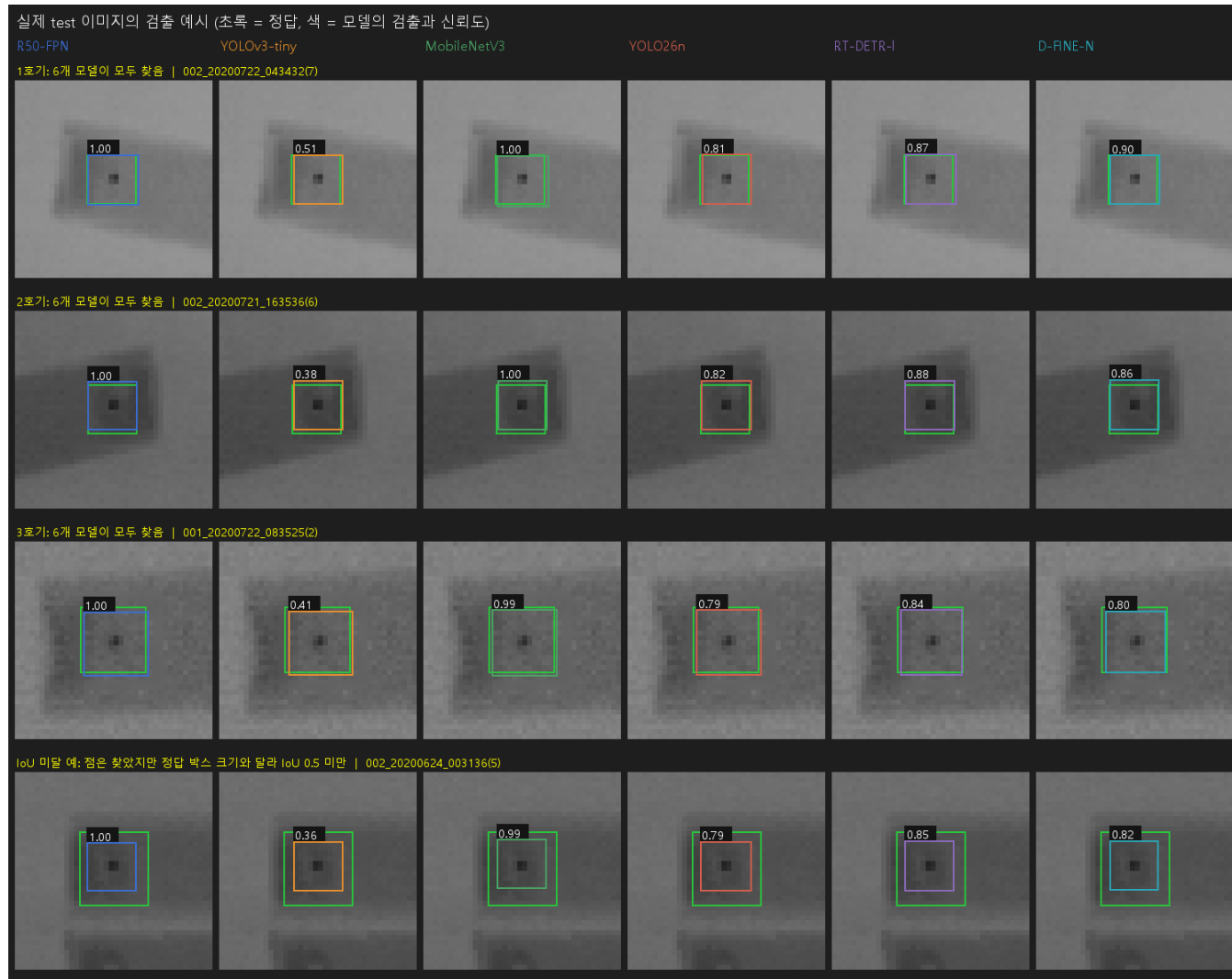
F1은 0.986~0.991, 정답 663개 중 6~9개를 놓쳤고 거의 전부 공식 라벨 이미지다

모델 (test, IoU 0.5)	F1	못 찾음(FN)	그중 공식 라벨 이미지
YOLOv3-tiny (베이스라인)	0.989	7	7
Faster R-CNN R50	0.989	7	7
Faster R-CNN MobileNetV3	0.986	9	8
YOLO26n	0.991	6	6
RT-DETR-I	0.988	8	7
D-FINE-N	0.989	7	7

- 공식 라벨만 보면 F1은 0.96대로 내려간다
- FP도 같은 수 → 같은 점을 **어긋난 박스**로 낸 것
- → 정말 놓친 것일까? 놓친 자리를 직접 확인했다

실제 검출 예시: 6개 모델이 같은 점을 찾았다

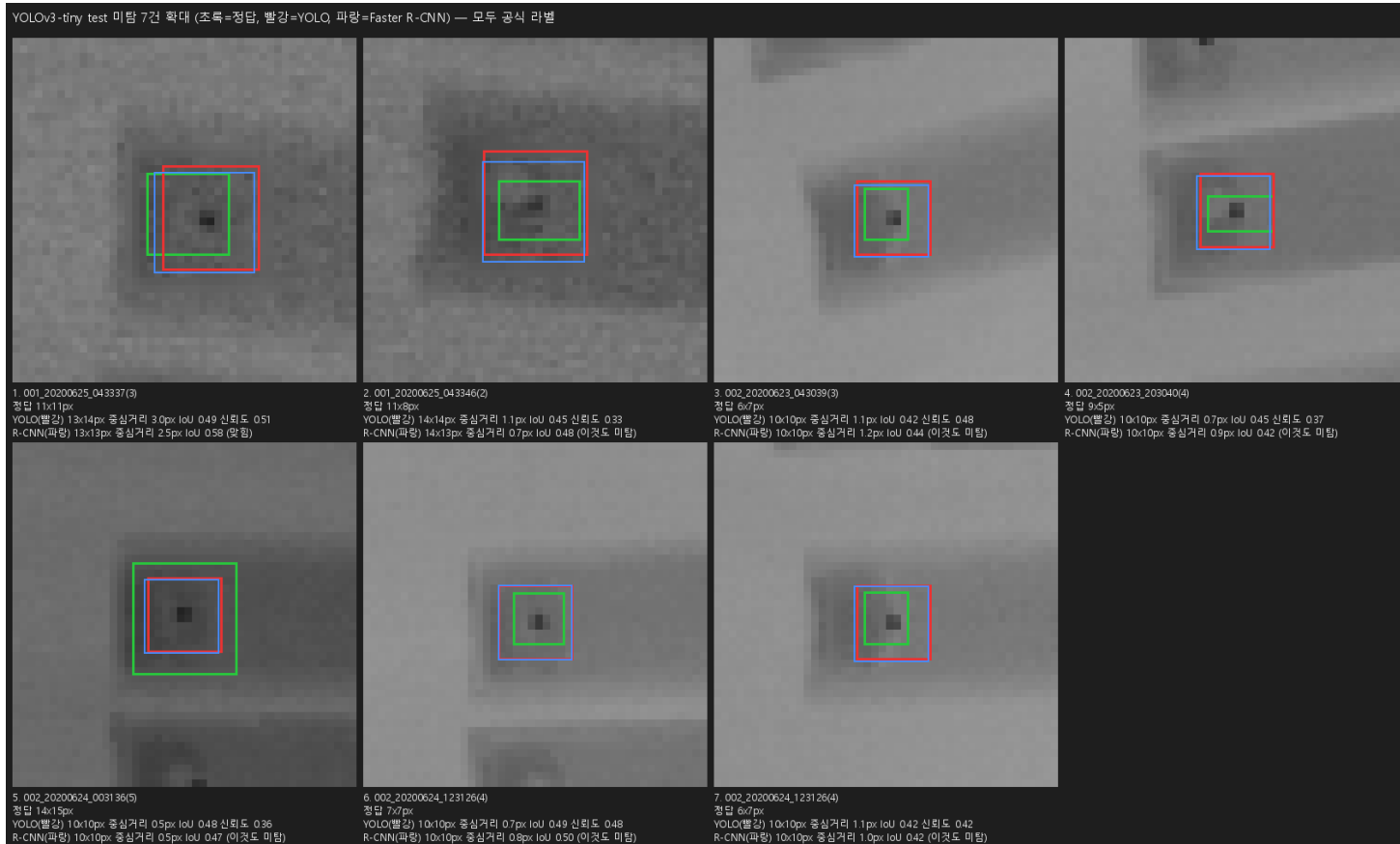
호기마다 한 장씩, 초록 = 정답, 색 = 모델의 검출과 신뢰도



- 1~3행: 6개 모델 모두 점을 정확히 감싼다
- 신뢰도 척도가 모델마다 다르다(YOLOv3-tiny 0.4~0.5, 나머지 0.8~1.0) → 임계값을 모델마다 val에서 따로 정한 이유
- 4행: 점은 찾았지만 정답 박스(초록)가 더 커서 IoU가 0.5에 못 미친다 → "못 찾음"으로 집계

원인: 공식 라벨의 박스 크기

놓친 것이 아니라 정답 박스의 크기가 달라서 IoU가 0.5에 못 미친 것이다

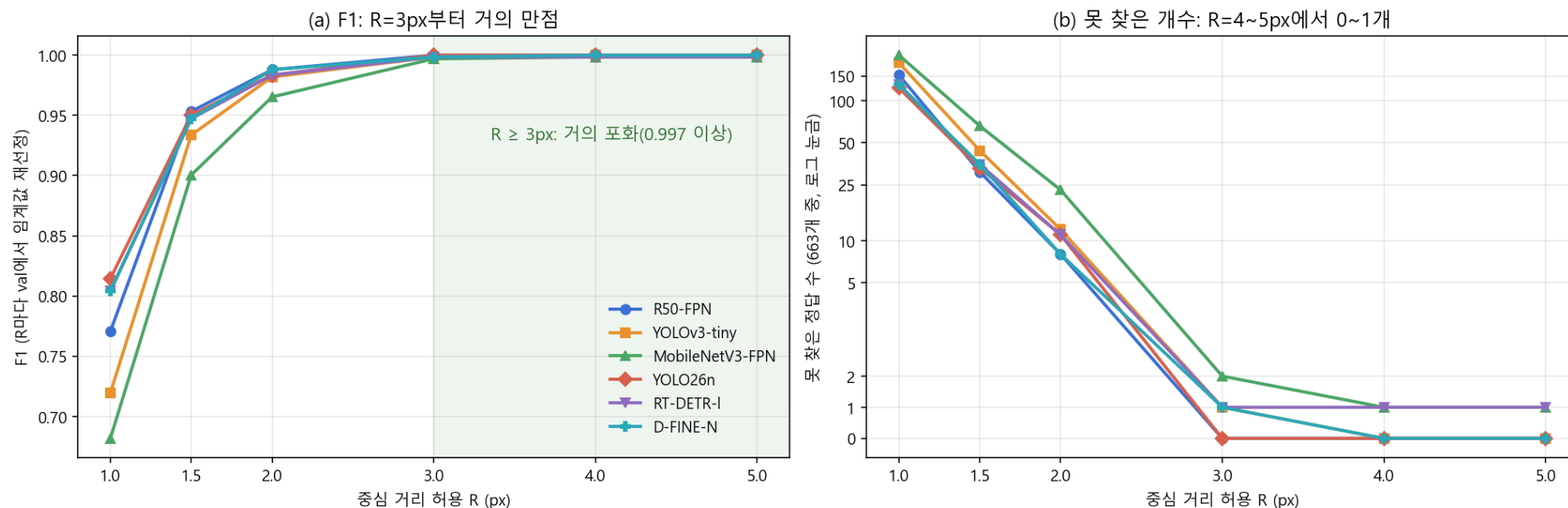


- 오류가 난 이미지는 거의 전부 공식 라벨(사람이 그려 크기가 제각각)
- 확대하면 모델 박스는 점을 정확히 감쌌(중심 거리 0.5~1.2px)
- 정답 박스가 더 작거나(6x7, 9x5) 더 큼(14x15)
- 팀 라벨은 고정 크기라 IoU가 잘 맞아 점수가 부풀려진다

해결: 중심 거리로 재채점

중심이 정답에서 $R(px)$ 이내면 찾은 것으로 보면 거의 다 찾는다

평가 v2(중심 거리 기준), 6개 모델 전부, test 정답 663개



- $R = 5px$ 로 재채점하면 못 찾은 것이 4개 모델은 **0개**, MobileNetV3·RT-DETR-L은 **1개**. $R = 3px$ 에서 F1은 **0.997~1.000**, $R = 4 \sim 5px$ 에서 **0.998~1.000**(포화)
- IoU 0.5는 사실상 "중심이 2px 이내인가"를 재고 있었다($R = 2px$ 에서 8~23개). $R = 1px$ 에서만 차이가 크다(YOLO26n 0.814, D-FINE-N 0.805, RT-DETR-L 0.804 > R50 0.771 > YOLOv3-tiny 0.719 > MobileNetV3 0.682)

중심 거리 R별 결과: 6개 모델 전부

F1 (못 찾은 정답 수). R마다 val에서 임계값을 다시 정해 test에 한 번 적용

모델 / R (px)	1	1.5	2	3	4	5
R50-FPN	0.771 (152)	0.953 (31)	0.988 (8)	1.000 (0)	1.000 (0)	1.000 (0)
YOLOv3-tiny	0.719 (186)	0.934 (44)	0.982 (12)	0.998 (1)	1.000 (0)	1.000 (0)
MobileNetV3-FPN	0.682 (211)	0.900 (66)	0.965 (23)	0.997 (2)	0.998 (1)	0.998 (1)
YOLO26n	0.814 (123)	0.950 (33)	0.983 (11)	1.000 (0)	1.000 (0)	1.000 (0)
RT-DETR-L	0.804 (130)	0.947 (35)	0.983 (11)	0.998 (1)	0.998 (1)	0.998 (1)
D-FINE-N	0.805 (130)	0.947 (35)	0.988 (8)	0.998 (1)	1.000 (0)	1.000 (0)

- R이 작을수록 모델 차이가 보이고(1px), R = 3px부터 포화한다. 2px는 IoU 0.5와 엄격도가 비슷해 함께 보고했다.

② 일반화와 robust 평가

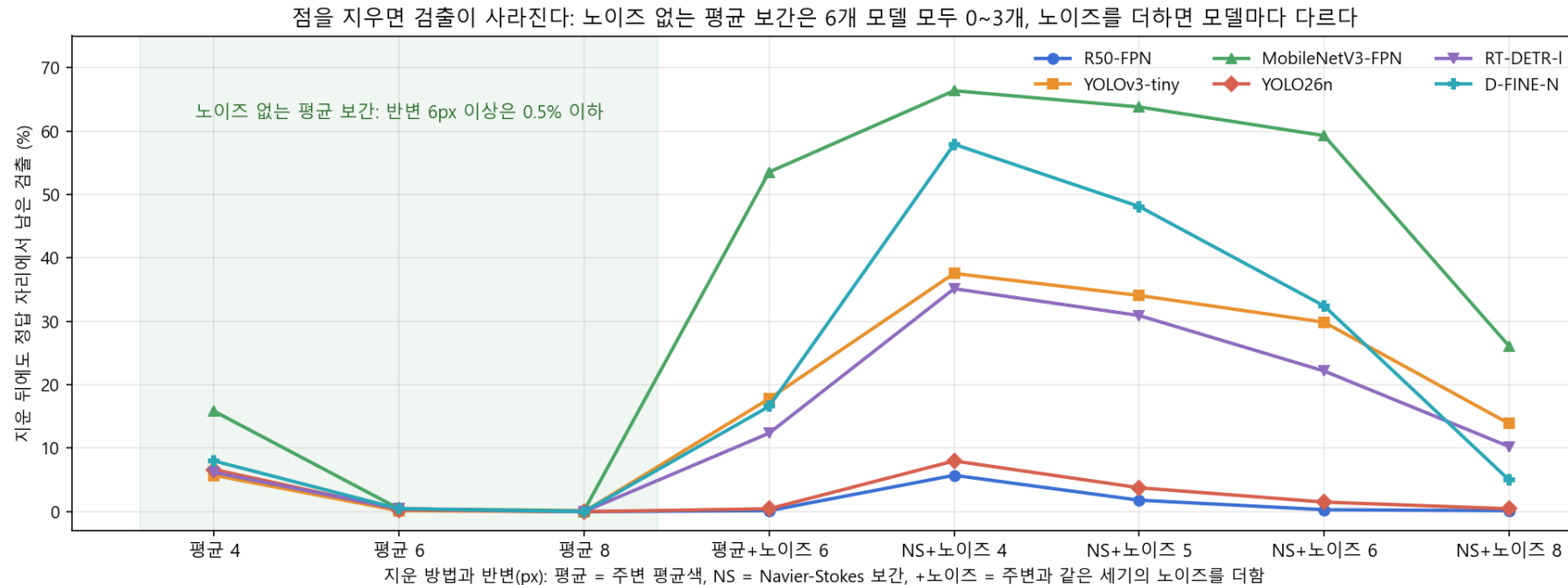
실제 데이터에서는 다 찾으니, 조건을 인위적으로 바꿔 모델을 시험한다

시험	묻는 것	방법
점 제거	모델이 결함의 점 자체를 보는가, 위치나 주변만 보는가	정답의 점을 보간으로 지운 이미지에 다시 적용
가짜 정상	결함이 없는 이미지에서 오경보를 내는가	점을 지운 이미지(결함 369장)를 가짜 정상으로 사용
합성 점	열은 점·다른 호기에서도 찾는가	작대기 안에만 점을 합성(진하기 4 x 크기 3), 시드 42
호기·해상도	특정 호기나 해상도에서 약한가	호기·해상도별로 나눠 평가

- 합성·가짜 정상은 **인위적인 이미지**라 실제 현장 성능이 아니라 **모델 간 상대 비교와 취약점 탐색**으로 해석한다

점 제거: 모델은 점 자체를 본다

결함의 점을 지우면 검출이 사라진다 → 위치만 보는 지름길이 아니다

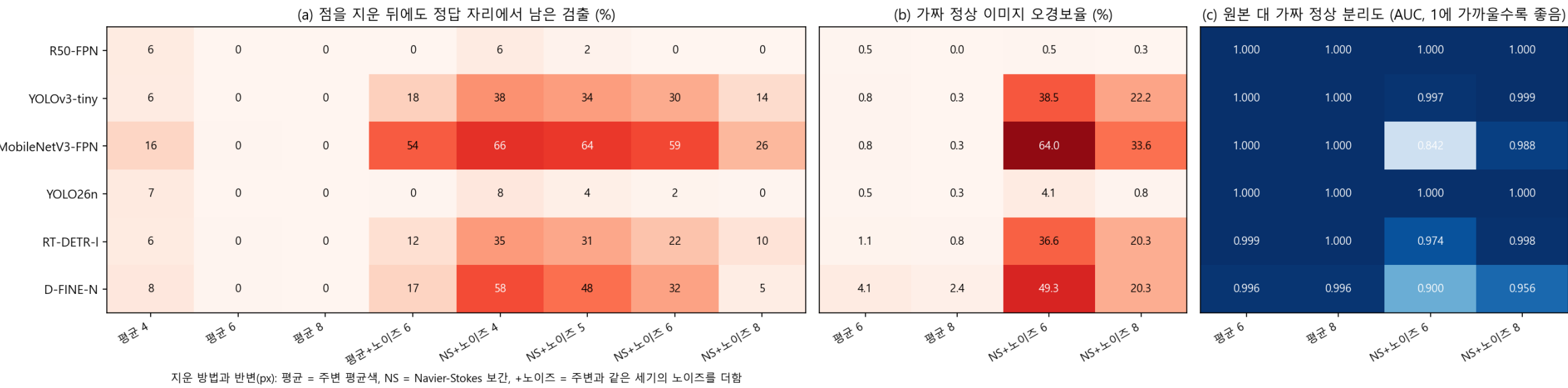


- test 정답 663개의 점을 보간으로 지우고 같은 모델을 다시 적용: 노이즈 없는 평균 보간으로 13x13 이상 지우면 **6개 모델 모두 0~3개만 검출**
- YOLOv3-tiny 등에 남는 반응은 보간에 더한 **노이즈 무늬** 때문 → 다음 질문: 노이즈 무늬에 속아 오경보를 내는가?

가짜 정상: 노이즈 보간에는 R50과 YOLO26n만 속지 않는다

정상 이미지가 없어 결함의 점을 지운 이미지로 오경보를 잤다

점 제거와 가짜 정상 시험: 6개 모델 (test 결함 369장, 점 663개, 모델마다 val 임계값)

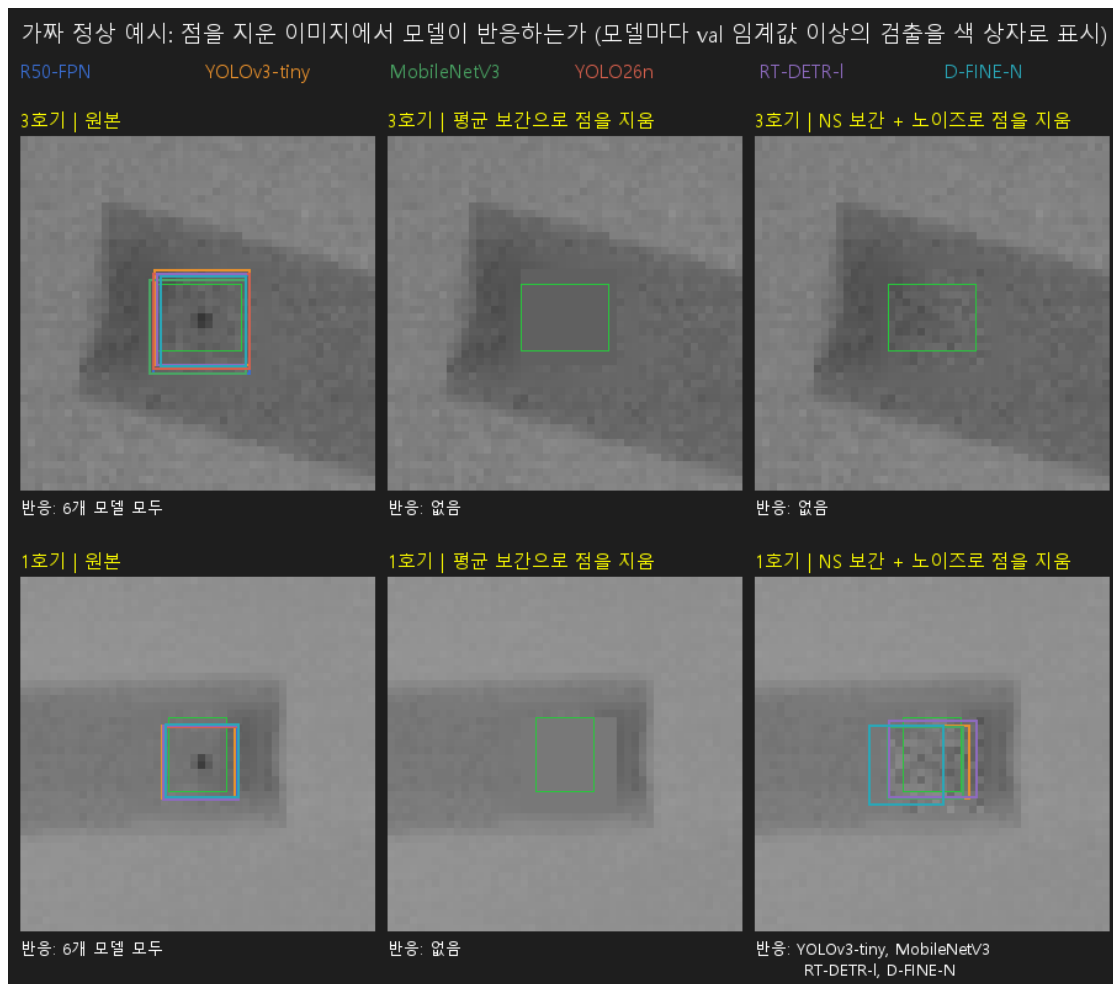


- NS+노이즈 6px 오경보율: **R50 0.5%, YOLO26n 4.1%**, RT-DETR-L 37%, YOLOv3-tiny 39%, D-FINE-N 49%, MobileNetV3 64%
- 오경보는 대부분 **지운 자리에서 발생**, 분리도(AUC)는 MobileNet 0.842·D-FINE 0.900이 가장 나쁨

가짜 정상은 진짜 정상 제품이 아니다. 오경보율은 범위로 해석한다.

가짜 정상 예시: 노이즈 무늬를 점으로 착각한다

평균 보간으로 지우면 반응이 없지만, 노이즈를 더하면 일부 모델이 반응한다

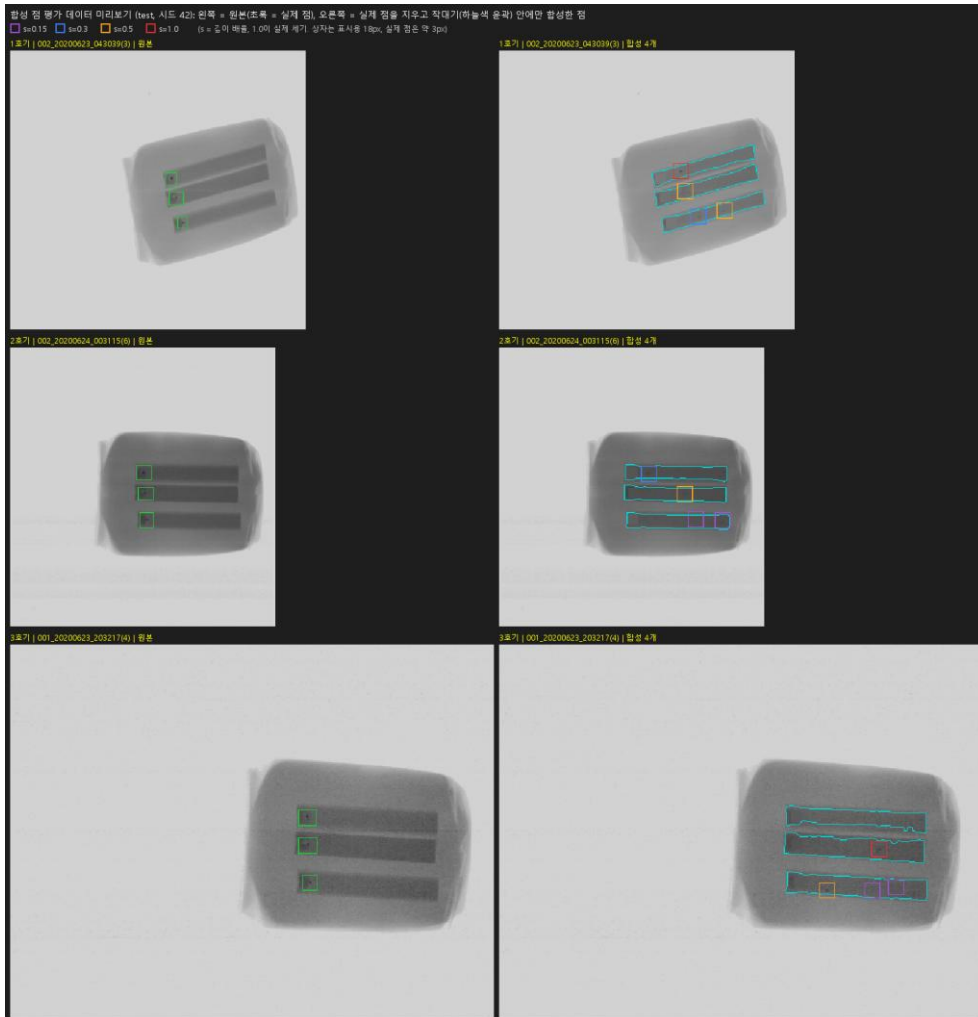


- 왼쪽 열: 원본 → 6개 모델이 모두 검출
- 가운데: 평균 보간으로 점을 지움 → 반응 없음(점이 단서)
- 오른쪽 아래(1호기): NS 보간 + 노이즈로 지운 자리의 **노이즈 무늬를 점으로 착각**해 YOLOv3-tiny, MobileNetV3, RT-DETR-L, D-FINE-N이 반응, R50과 YOLO26n은 반응 없음
- → 점의 모양보다 **질감**에 반응하는 모델이 있다

예시 두 장(집계는 앞 슬라이드의 369장). 이 예시의 노이즈는 이미지마다 새 난수(시드 0)로 만들어 집계용 이미지와 값이 다르다.

합성 점: 작대기 안에만, 시드 42로 모두가 같게

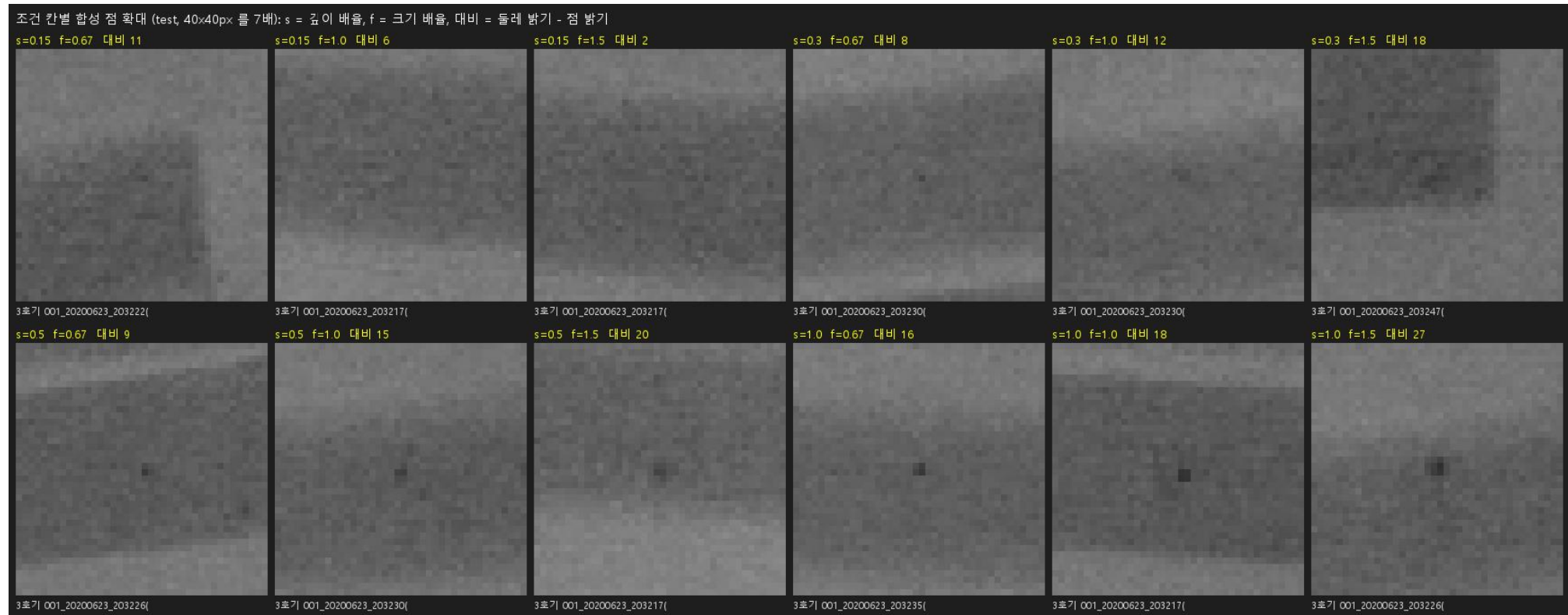
실제로는 농침이 사실상 없어서, 미탐지 조건을 직접 만들어 시험했다



- 실제 점을 지우고 영상에서 찾은 작대기(막대) 안쪽에만 val의 실제 점(투과율 조각)을 합성
- 점의 진하기(4단계) x 크기(3단계)를 바꿔 열린 점까지 시험
- test 바탕 386장, 점 1,436개. 실제 점 663개가 전부 검출된 막대에 닿음
- 시드 42 + 이미지 이름별 시드로 고정, 기준본과 --verify로 팀원 PC에서 같은 데이터인지 확인
- 합성은 실제보다 약 10%p 덜 현실적 → 절대 수치가 아닌 상대 비교

합성 점 조건별 확대

진하기(s)와 크기(f)를 바꾸면 대비가 달라지고, 대비가 낮을수록 눈에도 안 보인다

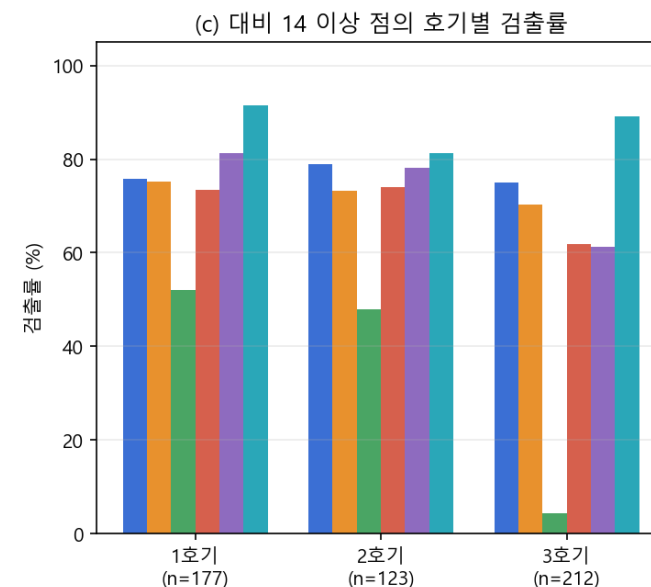
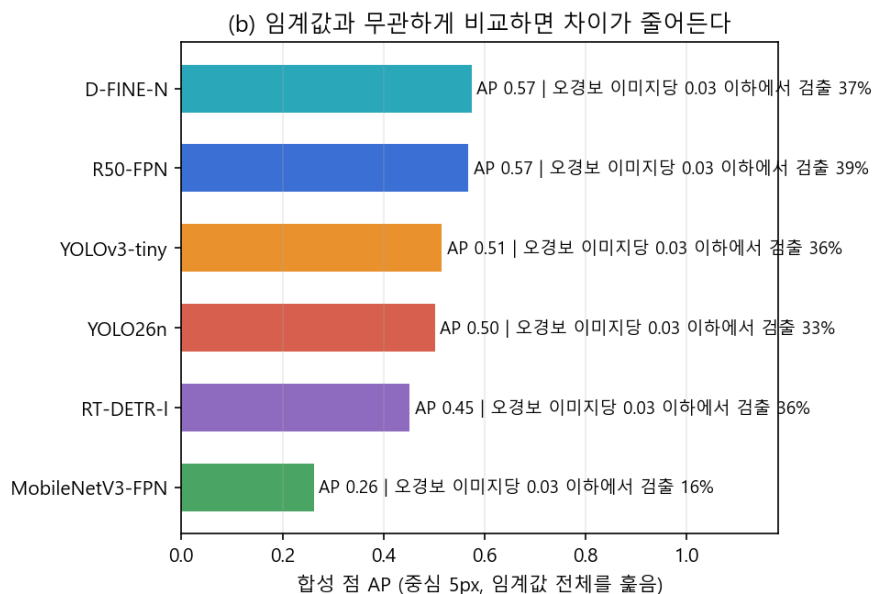
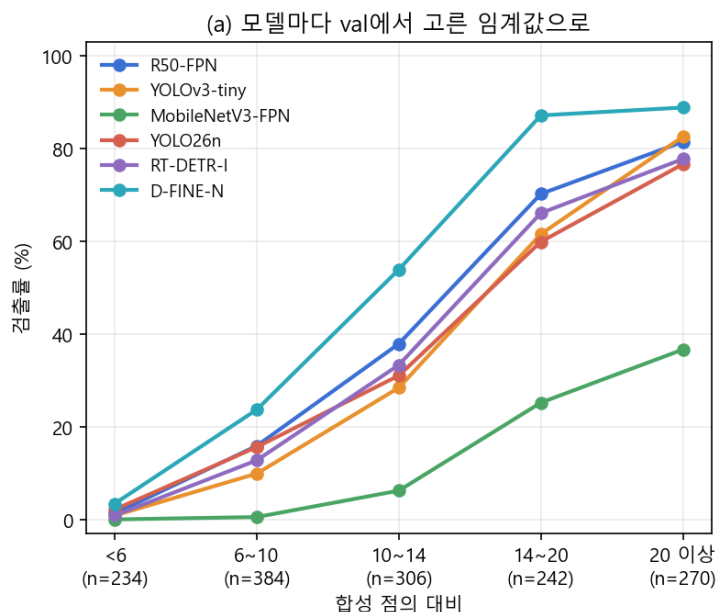


- s = 깊이 배율(1.0이 실제 점 세기), f = 크기 배율, 대비 = 둘레 밝기 - 점 밝기. 40x40px를 7배 확대
- 대비 6 미만은 거의 보이지 않고 모든 모델이 못 찾는다. 실제 test 점의 대비 중앙값은 22.8이라 $s=1.0$ 근처가 실제 수준이다

합성 점 결과: 열은 점이 미탐지 조건이다

val 임계값에서는 D-FINE-N이 앞서지만 대부분 느슨한 임계값 효과, 맞추면 차이가 줄어든다

합성 점(작대기 안, 시드 42) 평가: 6개 모델

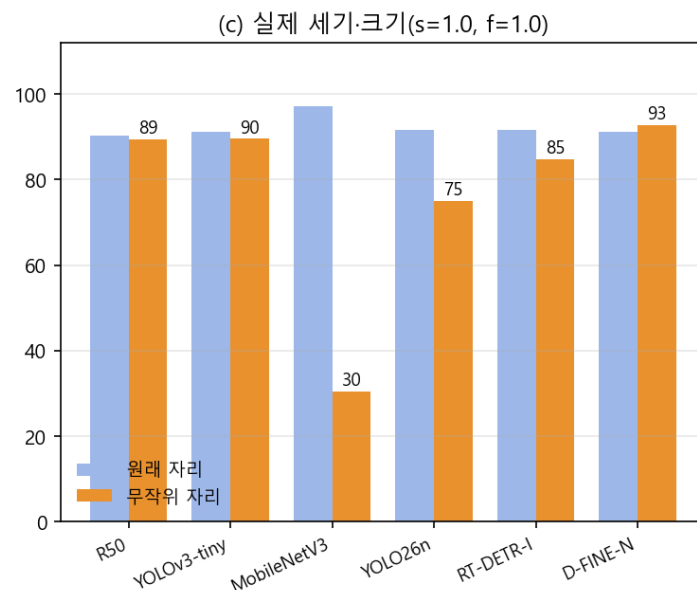
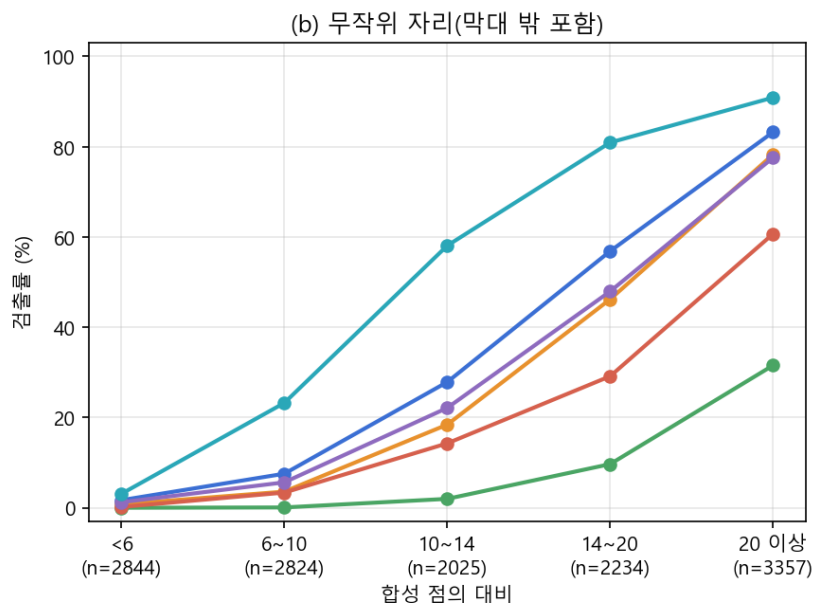
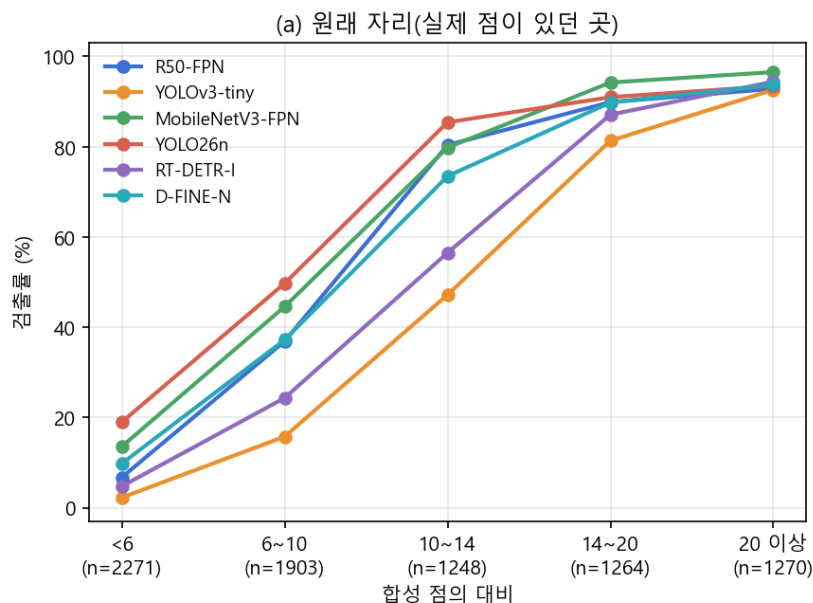


- 대비(둘레-점 밝기) 6 미만은 모든 모델이 거의 0%, 20 이상에서 약 77~89%
- 임계값과 무관한 합성 점 AP: **D-FINE-N 0.574** $\approx$ **R50 0.567** > YOLOv3-tiny 0.515 $\approx$ YOLO26n 0.501 > RT-DETR-L 0.451 > **MobileNetV3 0.262**
- 3호기(대비 14 이상): MobileNetV3 4%, YOLO26n·RT-DETR-L 약 61%, R50·YOLOv3-tiny·D-FINE-N은 호기 차이가 작음

합성 점 1차: 위치를 바꿔 시험

무작위 자리에서 MobileNetV3는 크게, YOLO26n·RT-DETR-I은 일부 떨어진다

합성 점 1차(원래 자리 663개 / 무작위 자리 1,107개), 6개 모델 전부, 모델마다 val 임계값



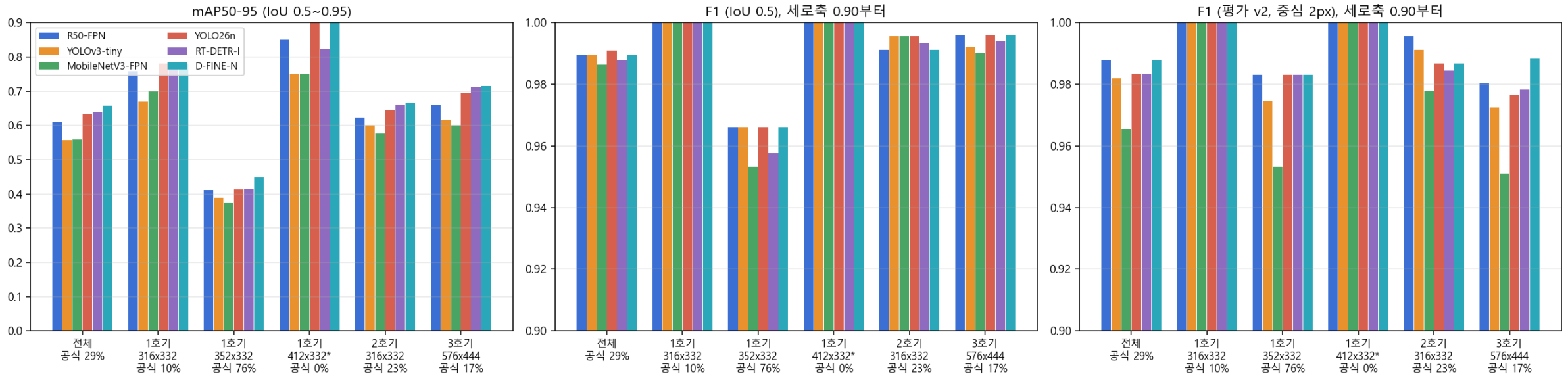
- 실제 세기·크기($s=1.0$, $f=1.0$): 원래 자리는 6개 모델 모두 90~97%, 무작위 자리는 D-FINE-N 93%, YOLOv3-tiny 90%, R50 89%, RT-DETR-I 85%, YOLO26n 75%, **MobileNetV3 30%**
- 원래 → 무작위 하락: MobileNetV3 -67%p, YOLO26n -17%p, RT-DETR-I -7%p, R50·YOLOv3-tiny·D-FINE-N은 거의 없음. 무작위 자리의 열린 점(대비 10~14)은 **D-FINE-N 58%**로 가장 많이 찾는다

1차 합성은 막대 밖 자리도 포함하고 합성 점의 사각형 자국이 교란 요인이다. 같은 시드로 기존 3개 모델 값이 모두 재현됐다. 합성 점이므로 상대 비교로만 쓴다.

호기·해상도: 뚜렷한 효과는 확인되지 않았다

가장 낮은 그룹(test)은 공식 라벨 박스가 76%여서 IoU 점수가 낮을 뿐이다

호기·해상도별 성능, 6개 모델 (test, 모델별 val 고정 임계값). x축 아래 '공식 N%' = 그 그룹의 공식 라벨 박스 비율. *412x332는 박스 2개뿐



- 해상도별 공식 라벨 박스 비율(test 기준)이 10~76%로 달라 라벨 구성이 섞인다. 공식 라벨 이미지끼리만 비교하면 6개 모델의 해상도별 F1이 0.944~0.983으로 좁아지고 구간이 겹친다
- 6개 모델 모두 가장 낮은 그룹은 1호기·352x332(F1 0.953~0.966)다. 그룹별 표본이 작다(공식 라벨 이미지 호기별 20~36장). 합성 점에서 보인 3호기 약점은 **모델에 따라 다르다**(앞 슬라이드)

③ 종합: 모델별 성적표

한 모델이 모든 항목에서 1등은 아니다

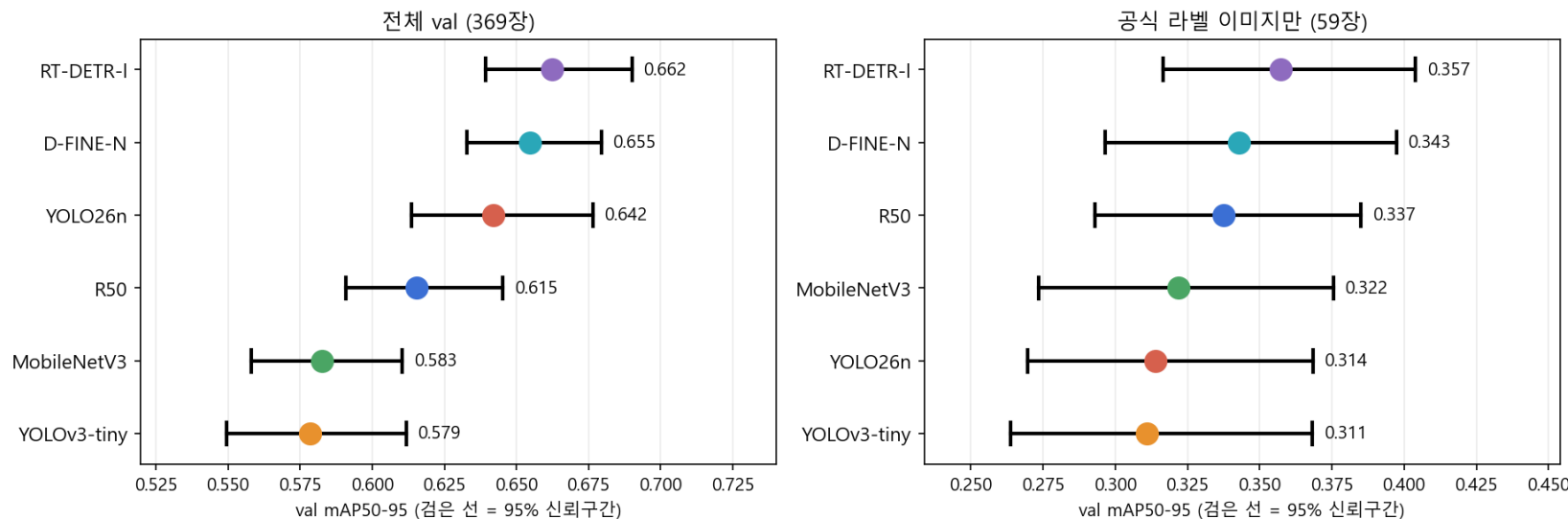
모델	test F1 (IoU 0.5)	val mAP50-95 (전체 / 공식만)	가짜 정상 오경보 (NS+노이즈 6)	합성 점 AP	3호기 합성 점 (대비 14 이상)	파라미터
D-FINE-N	0.989	0.655 / 0.343	49.3%	0.574	89%	3.7M
RT-DETR-I	0.988	0.662 / 0.357	36.6%	0.451	61%	32.8M
Faster R-CNN R50	0.989	0.615 / 0.337	0.5%	0.567	75%	41.4M
YOLO26n	0.991	0.642 / 0.314	4.1%	0.501	62%	2.5M
YOLOv3-tiny	0.989	0.579 / 0.311	38.5%	0.515	70%	8.7M
Faster R-CNN MobileNetV3	0.986	0.583 / 0.322	64.0%	0.262	4%	19.0M

- 점 제거는 6개 모델 모두 0~3개만 남아 차이가 없다. test F1은 모두 포함다.

베스트 모델: D-FINE-N

val 동물 선두 + 미탐지 강건성 최상 + 작다(3.7M)

RT-DETR-I과 D-FINE-N이 동물 선두 (전체 val). 공식 라벨만 보면 구간이 넓어 R50도 동물



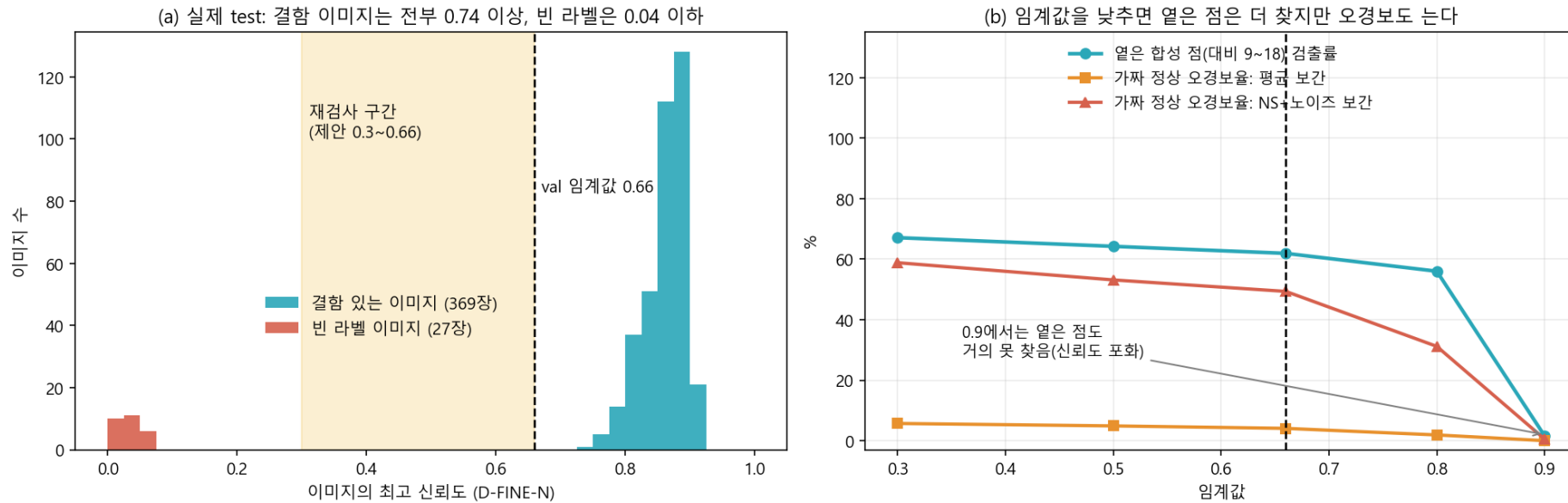
- **약점:** 노이즈 보간 오경보 49%, 신뢰도가 0.9 근처에서 포화
- **대안:** 오경보 우선이면 R50 / YOLO26n, CPU 전용이면 YOLOv3-tiny
- '미탐지 우선'은 결과를 본 뒤 정한 우선순위라 보고서에 그대로 밝힌다

- ① **val mAP50-95:** val 이미지 복원추출 1,000회로 비교하니 **RT-DETR-I 0.662와 D-FINE-N 0.655가 동물 선두**(차이 -0.008 , 95% 구간 $-0.016 \sim 0.000$). R50·YOLO26n은 유의하게 낮고, 공식 라벨만 보면 R50도 동물
- ② **동물이면 강건성:** 합성 점 AP와 3호기 검출은 D-FINE-N 우세(0.574 대 0.451, 89% 대 61%), 가짜 정상 오경보는 RT-DETR-I 우세(37% 대 49%) → 놓침이 더 비싸다고 보아 **미탐지를 우선**
- ③ **크기:** 3.7M(15MB)로 RT-DETR-I(32.8M)의 약 1/9

현장 활용: 3단 판정과 재검사 구간

실제 test에서는 결함 369장(≥ 0.748)과 빈 라벨 27장(≤ 0.04)이 완전히 갈린다

최종 모델 D-FINE-N의 임계값 운영 근거 (합성·가짜 정상은 참고용)



- 불합격 최고 신뢰도 ≥ 0.66 (val 임계값) | 재검사 0.3 ~ 0.66 | 합격 < 0.3
- 하한을 0.3으로 낮추면 오픈 점 검출 61.9% $\rightarrow$ 67.1%, 대신 가짜 정상 오경보 4.1% $\rightarrow$ 5.7%(평균 보간), 49% $\rightarrow$ 59%(노이즈 보간)
- 오경보가 비싼 현장은 재검사 구간의 이득이 작다

모델 선택 가이드와 처리 속도

촬영 간격(중양값 4초)보다 모든 모델이 빠르다

상황	권장	이유
기본(최종 모델)	D-FINE-N	val 동률 선두, 미탐지 강건성 최상, 3.7M. 노이즈 오경보 49%
오경보가 중요한 현장	Faster R-CNN R50 또는 YOLO26n	노이즈 오경보 0.5%~4.1%. R50은 느리고 YOLO26n은 3호기 합성 점에 약함
CPU 전용 / 최경량	YOLOv3-tiny	CPU 4스레드 약 15 FPS(6개 중 가장 빠름), GPU FP16에서 258 FPS, 정확도 손실이 작음
비권장	MobileNetV3	위치 의존, 합성 점 AP 0.262, 노이즈 오경보 34~64%

처리 속도 FPS (3회 중 최고)	YOLOv3-tiny	YOLO26n	MobileNetV3	D-FINE-N	RT-DETR-I	R50
GPU FP32	207	104	102	48	32	20
GPU FP16	258	92	88	42	43	35
CPU 4스레드	15	9.2	10	9.7	2.0	0.9
CPU 1스레드	6.5	11	8.4	7.4	2.0	1.1

같은 조건(전처리 제외, 모델 연산+후처리, 배치 1)으로 3회 재어 최고값 사용. GPU FP32는 3회 ±15% 이내로 안정적, GPU FP16-CPU는 뒤쪽 패스에서 열.전력 제한으로 최대 11배 느려졌다. FP16은 가중치·입력을 half로 바꿨고 정확도(F1)는 재지 않았다.

차별점: 점수를 믿게 만드는 검증

못 찾은 것의 원인을 확인하고, 일반화까지 시험해 모델을 골랐다

- **오류의 원인 규명:** 놓친 것이 아니라 라벨 박스 크기 차이임을 확대 확인하고, **중심 거리 평가**로 재채점
- **점 제거 시험:** 결함을 지워 모델이 점 자체를 보는지 확인 → 지름길 우려 해소
- **가짜 정상과 합성 점:** 정상 이미지가 없는 데이터에서 오경보·미탐지 조건(열은 점, 위치, 호기)을 정량화
- **임계값과 무관한 비교:** AP·오경보 예산으로 임계값 효과를 걷어냄(D-FINE-N의 우위가 임계값 효과임을 확인)
- **공유 가능한 평가 데이터:** 시드 42 합성 데이터 + 기준본 검증으로 팀 모두가 같은 시험을 재현
- **분석을 조치로:** 취약점이 모델 선택 가이드와 재검사 구간으로 이어짐

코드 구성과 재현성

전처리 · 학습 · 추론 · 평가 · 분석을 폴더별로 분리하고 실행 명령을 문서화했다

- 환경: conda KAMP(Python 3.10), torch 2.6, 한글 경로 용 PYTHONUTF8=1, requirements.txt
- src/yolov3/: 베이스라인(호환 수정본)
- src/minyeop/: Faster R-CNN, MobileNet, YOLOv3-tiny, 팀원 모델 평가, 분석(점 제거, 가짜 정상, 합성 점, 호기 통계)
- scripts/synth_eval.py: 시드 42 합성 평가 데이터 생성기
- docs/: 지표, 실험 결과 표, 분석 해석
- 평가 규칙: 임계값은 **val에서 고정**, test는 한 번만
- 데이터 버전: manifest.csv의 sha256으로 확인
- 합성 평가 데이터: 기준본과 --verify로 비교(위치·조건·픽셀 해시)
- 모델 선정: val 부트스트랩 구간으로 "동률"을 객관적으로 판정
- 모든 수치의 출처 파일을 docs/와 runs/에 보존(예측 JSON은 제외)

한계와 향후 과제

숨기지 않고 적는다

- 합성 점과 가짜 정상은 **인위적인 이미지**다. 실제 현장 성능이 아니라 상대 비교와 취약점 탐색으로만 해석한다
- 강건성 시험은 **test 이미지**로 먼저 수행했다. 모델 선정에는 val 지표만 썼지만, 선정 근거로 쓰려면 val로 반복해야 한다
- '미탐지 우선'이라는 우선순위는 결과를 본 뒤 정했다. 오경보 우선이면 선정이 바뀐다
- 표시(색 사각형) 제거의 **흔적**이 단서가 되는지는 검증하지 못했다
- 학습 시드가 하나이고, 팀원 모델은 학습 조건이 우리와 다르다(임계값은 우리 규칙으로 재선정)
- 속도는 노트북에서 측정했고 실행마다 변동이 크다. 현장 장비 사양과 재검사 비용은 알지 못한다
- 향후: 흔적 검증(원본 BMP 2x2 실험), 실제 정상 제품 이미지 확보, val 반복, 합성 점 대조군($s=0$)

결론

- 6개 모델을 같은 조건으로 학습·평가했고 F1은 0.99 안팎이었다. 몇 개 못 찾은 것은 **공식 라벨의 박스 크기** 때문이었고 **중심 거리**로 재채점하면 거의 다 찾았다
- 일반화·robust 평가로 진짜 차이가 드러났다: **열은 결함, 노이즈 무늬 오경보, 위치 의존(MobileNetV3가 가장 크고 YOLO26n·RT-DETR-I도 일부)**
- 종합해서 **D-FINE-N**을 베스트 모델로 선정: val 동률 선두, 미탐지 강건성 최상, 작다. 약점(노이즈 오경보)은 임계값·재검사 구간과 대안 모델로 관리한다
- 공유 가능한 합성 평가 데이터와 검증 코드로 **팀 모두가 같은 시험을 재현한다**